

Decision Tree Algorithm

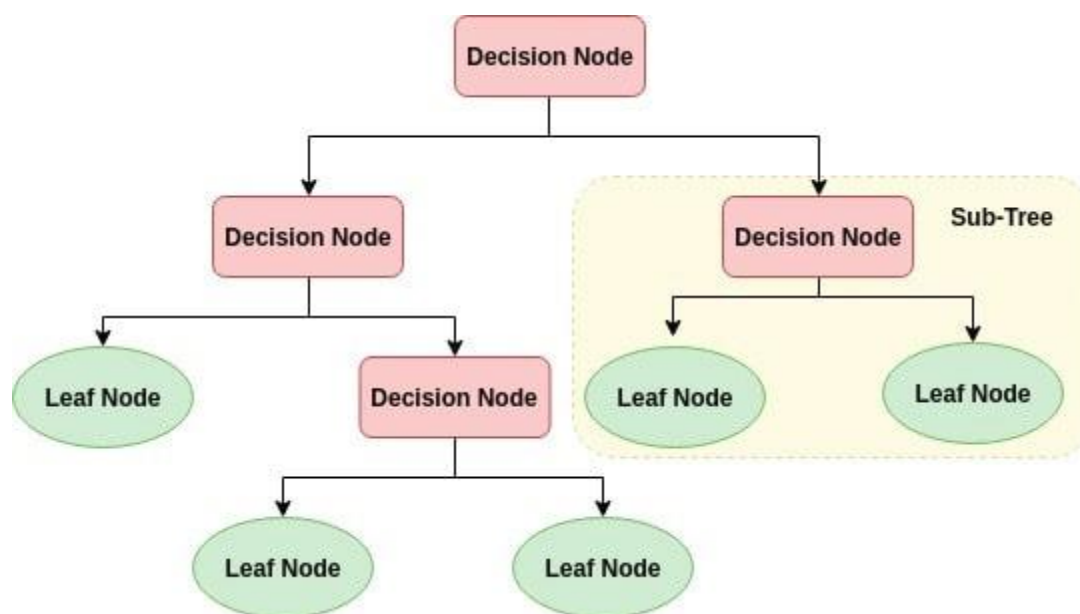
A decision tree is a flowchart-like tree structure where an internal node represents feature (or attribute), the branch represents a decision rule, and each leaf node represents the outcome.

The topmost node in a decision tree is known as the root node. It learns to partition based on the attribute value. It partitions the tree in recursively manner call recursive partitioning.

This flowchart-like structure helps you in decision making.

It's visualization like a flowchart diagram which easily mimics the human level thinking.

That is why decision trees are easy to understand and interpret.



Decision Tree is a white box type of ML algorithm. It shares internal decision-making logic, which is not available in the black box type of algorithms such as Neural Network.

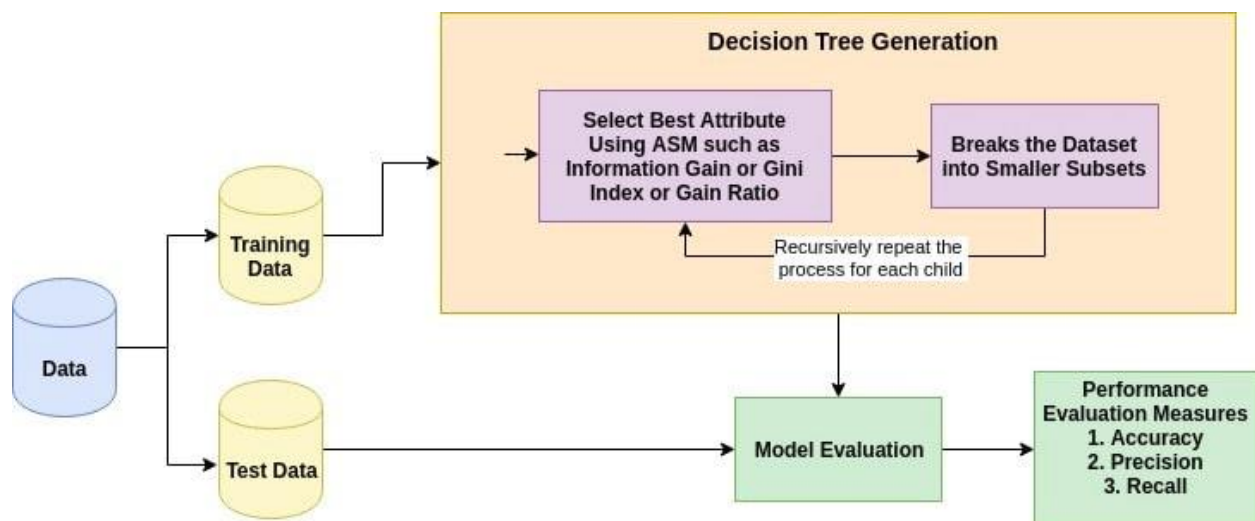
Its training time is faster compared to the neural network algorithm. The time complexity of decision trees is a function of the number of records and number of attributes in the given data.

The decision tree is a distribution-free or non-parametric method, which does not depend upon probability distribution assumptions.

Decision trees can handle high dimensional data with good accuracy.

The basic idea behind any decision tree algorithm is as follows:

1. Select the best attribute using Attribute Selection Measures(ASM) to split the records.
2. Make that attribute a decision node and breaks the dataset into smaller subsets.
3. Starts tree building by repeating this process recursively for each child until one of the conditions will match:
 - All the tuples belong to the same attribute value.
 - There are no more remaining attributes.
 - There are no more instances.



Attribute Selection Measures

Attribute selection measure is a heuristic for selecting the splitting criterion that partition data into the best possible manner. It is also known as splitting rules because it helps us to determine breakpoints for tuples on a given node. ASM provides a rank to each feature(or attribute) by explaining the given dataset.

Most popular selection measures are Information Gain, Gain Ratio, and Gini Index.

Information Gain

Shannon invented the concept of entropy, which measures the impurity of the input set. In physics and mathematics, entropy referred as the randomness or the impurity in the system. In information theory, it refers to the impurity in a group of examples. Information gain is the decrease in entropy. Information gain computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values. ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

$$\text{Info}(D) = - \sum_{i=1}^m p_i \log_2 p_i$$

Where, p_i is the probability that an arbitrary tuple in D belongs to class C_i .

$$\text{Info}_A(D) = \sum_{j=1}^V \frac{|D_j|}{|D|} \times \text{Info}(D_j)$$

Where,

- $\text{Info}(D)$ is the average amount of information needed to identify the class label of a tuple in D .
- $|D_j|/|D|$ acts as the weight of the j th partition.
- $\text{Info}_A(D)$ is the expected information required to classify a tuple from D based on the partitioning by A .

The attribute A with the highest information gain, $\text{Gain}(A)$, is chosen as the splitting attribute at node $N()$.

Gain Ratio

Information gain is biased for the attribute with many outcomes. It means it prefers the attribute with many distinct values. For instance, consider an attribute with a unique identifier such as `customer_ID` has zero $\text{info}(D)$ because of pure partition. This maximizes the information gain and creates useless partitioning.

C4.5, an improvement of ID3, uses an extension to information gain known as the gain ratio. Gain ratio handles the issue of bias by normalizing the information gain using Split Info.

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

Where,

- $|D_j|/|D|$ acts as the weight of the j th partition.
- v is the number of discrete values in attribute A .

The gain ratio can be defined as

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}$$

The attribute with the highest gain ratio is chosen as the splitting attribute

Gini index

Another decision tree algorithm CART (Classification and Regression Tree) uses the Gini method to create split points.

$$\text{Gini}(D) = 1 - \sum_{i=1}^m P_i^2$$

Where, p_i is the probability that a tuple in D belongs to class C_i .

The Gini Index considers a binary split for each attribute. You can compute a weighted sum of the impurity of each partition. If a binary split on attribute A partitions data D into D_1 and D_2 , the Gini index of D is:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

In case of a discrete-valued attribute, the subset that gives the minimum gini index for that chosen is selected as a splitting attribute. In the case of continuous-valued attributes, the strategy is to select each pair of adjacent values as a possible split-point and point with smaller gini index chosen as the splitting point.

$$\Delta \text{Gini}(A) = \text{Gini}(D) - \text{Gini}_A(D).$$

The attribute with minimum Gini index is chosen as the splitting attribute.

Pros

- Decision trees are easy to interpret and visualize.
- It can easily capture Non-linear patterns.
- It requires fewer data preprocessing from the user, for example, there is no need to normalize columns.
- It can be used for feature engineering such as predicting missing values, suitable for variable selection.
- The decision tree has no assumptions about distribution because of the non-parametric nature of the algorithm.

Cons

- Sensitive to noisy data. It can overfit noisy data.
- The small variation(or variance) in data can result in the different decision tree. This can be reduced by bagging and boosting algorithms.
- Decision trees are biased with imbalance dataset, so it is recommended that balance out the dataset before creating the decision tree.